



A Course End Project Report on

EXPENSE TRACKER

Python Programming Laboratory

Submitted by

Varala Abhishek

(25881A0566)

Usham Manoj

(25881A0597)

Course Facilitator

Ms. P.S. Madhavi

Assistant Professor

Department of Computer Science and Engineering Vardhaman

College of Engineering, Hyderabad

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled “**Expense Tracker**” is carried out by **Varala.Abhishek** and **Usham Manoj Reddy** with Roll Numbers **25881A0566** and **25881A097** towards **python programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty

Ms P.S.Madhavi
Assistant Professor, CSE

Vardhaman college of
Engineering

**Signature of
the HOD**

Dr .Y.Vijayalata
HOD,
Dean(Student
Mentoring),
CSE,
Vardhaman
college of
Engineering

Contents

Abstract	4
1 Introduction	5
2 Literature Review	5
3 Objectives	6
4 Methodology	6
4.1 Architectural Diagram	6
4.2 Hardware and Software Requirements	7
4.3 Working Principle	7
4.4 Software Implementation	7
5 Results and Discussion	10
6 Mapping POs, PSOs and SDGs	12
6.1 Program Outcomes (POs) Mapping	12
6.2 Program Specific Outcomes (PSOs) Mapping	13
6.3 Sustainable Development Goals (SDGs) Mapping	14
7 Conclusion	15
8 Bibliography	15

Abstract

The Expense Tracker is a Python-based personal expense management application developed to help users record, view, and analyse day-to-day expenditures in a simple and structured manner. The project addresses a common practical problem: expenses are often recorded informally or forgotten, making it difficult to understand where money is being spent. The application stores each expense as a Python dictionary inside a list and provides a menu-driven interface for performing basic expense-management operations.

The system supports four major operations: displaying all recorded expenses, calculating the overall expenditure, filtering expenses by category, and adding a new expense. Each expense contains an ID, title, amount, category, date, payment method, and note. Category-wise search is case-insensitive, while the total expense is calculated dynamically from the stored records. The program uses fundamental Python concepts such as lists, dictionaries, functions, loops, conditional statements, input handling, type conversion, and string operations.

The project demonstrates how a small command-line application can transform raw financial entries into useful information. It is intentionally lightweight and suitable as an academic course-end project, while its modular structure also provides a foundation for future extensions such as editing and deleting records, monthly summaries, budget limits, charts, file/database storage, authentication, and a graphical or web interface.

Keywords: Expense Tracker, Python, Personal Finance, Expense Management, Lists, Dictionaries, Functions, CLI Application

1 Introduction

Personal financial awareness is an important part of everyday decision-making, especially for students and young professionals who manage expenses such as food, transportation, education, utilities, entertainment, and other personal needs. When expenses are not recorded in a structured manner, it becomes difficult to calculate the total amount spent or identify the categories that consume the largest share of available money.

The Expense Tracker project is designed as a simple software solution to this problem. It provides a menu-driven command-line application in Python where each transaction is represented as a structured record.

Instead of maintaining separate variables for every expense, the project uses a list of dictionaries, making it easier to store multiple records and access their fields using meaningful keys.

The current implementation focuses on four essential functions. The first displays all stored expenses with their complete details. The second calculates the total expenditure. The third searches for expenses belonging to a user-specified category and also calculates the total for that category. The fourth accepts a new expense from the user and appends it to the existing list. A continuous menu allows these operations to be repeated until the user selects the Exit option.

The initial dataset contains five sample transactions covering Food & Groceries, Transportation, Education, Bills & Utilities, and Entertainment. The total of these sample expenses is ₹3,649. This provides a small but meaningful dataset for testing the main functions and demonstrating the working of the applic

2 Literature Review

Expense-management systems are commonly implemented using spreadsheets, mobile applications, web applications, and database-backed personal finance platforms. Spreadsheet-based approaches provide flexibility but require manual formulas, consistent data entry, and user maintenance. Mobile and web applications generally provide richer features such as authentication, charts, notifications, cloud synchronization, and automated categorisation, but they also introduce additional design and implementation complexity.

For an academic course-end project, a lightweight Python implementation is useful because it clearly demonstrates the fundamental programming concepts behind an expense-management system without depending on a large external framework. Python lists provide ordered storage of multiple records, dictionaries represent structured transactions, functions divide the program into reusable modules, and loops and conditions implement searching, aggregation, and menu control.

The present project therefore focuses on the core logic of expense tracking rather than advanced financial services. The design can be viewed as a foundation on which more sophisticated features can later be built, including persistent storage, data visualisation, budget analysis, export facilities, and a graphical user interface.

3 Objectives

- To design and develop a simple Python-based application for recording personal expenses.
- To represent individual expense transactions using dictionaries and maintain them using a list.
- To provide a menu-driven interface for repeated user interaction.
- To display complete details of all recorded expenses in a readable format.
- To calculate the total expenditure dynamically from the stored records.
- To filter expenses by category using case-insensitive comparison.
- To calculate the total expenditure for a selected category.
- To allow users to add new expense records during program execution.
- To apply modular programming through separate functions for major operations.
- To provide a clear foundation for future features such as editing, deletion, budgets, reports, and persistent storage.

4 Methodology

The project follows a simple modular development methodology. The problem is first divided into data representation, display, calculation, filtering, insertion, and menu-control requirements. A Python list is selected as the main collection, while dictionaries are used to represent individual transactions. Each operation is then implemented as an independent function, reducing duplication and making the program easier to understand and maintain.

The application is developed and tested as a command-line program. Sample data is used initially to verify display and calculation operations. Additional records can be entered through the Add Expense option. The program is executed repeatedly through a while loop so that the user can perform multiple operations within a single run.

4.1 Architectural Diagram

Module	Responsibility	Main Python Concept
Expense Data Store	Stores all transaction records	List + Dictionary
Display Module	Shows every recorded expense	for loop, dictionary access
Total Calculator	Computes overall expenditure	loop + arithmetic
Category Module	Filters and totals a selected category	input + condition
Add Expense Module	Collects and stores a new record	input + append()
Menu Controller	Connects user choices to functions	while + if/elif

Figure 1: Logical architecture of the Expense Tracker. The user interacts with the menu controller, which invokes the required functional module. All modules operate on the central expense list, and results are printed to the command-line interface.

4.2 Hardware and Software Requirements

Category	Requirement
Processor	Intel/AMD processor or equivalent
RAM	Minimum 4 GB recommended
Storage	At least 100 MB free space for source code and Python environment
Operating System	Windows, Linux, or macOS
Programming Language	Python 3.x
Development Environment	Any Python-compatible IDE or text editor
Libraries	Python standard library only for the supplied implementation
Interface	Command-Line Interface (CLI)

4.3 Working Principle

At program start, a predefined list named `expenses` is created. Each element is a dictionary containing seven fields: `id`, `title`, `amount`, `category`, `date`, `payment_method`, and `note`. This structure keeps related information together and allows individual fields to be accessed using readable keys.

1. The program displays the main menu and waits for the user's choice.
2. For choice 1, `show_expenses()` traverses the complete list and prints every field of each record.
3. For choice 2, `total_expenses()` starts a running total at zero and adds each expense amount.
4. For choice 3, `category_expenses()` accepts a category, compares it using `lower()`, displays matching records, and calculates their subtotal.
5. For choice 4, `add_expense()` collects the required fields, creates a dictionary, and appends it to the `expenses` list.
6. For choice 5, the program prints a closing message and terminates the while loop.
7. Any other input is treated as invalid and the menu is shown again.

The implementation uses direct in-memory storage. Therefore, records added during one execution are available while the program is running but are not permanently stored after the program exits. This is an intentional limitation of the current version and an important area for future enhancement.

4.4 Software Implementation

The following is the complete Python implementation supplied for the Expense Tracker project. It contains the sample dataset, expense display, total calculation, category filtering, expense insertion, and menu-driven execution. The code is included in this section of the report and is not listed line-by-line in the Contents page.

```
expenses = [
    {
        "id": 1,
```

```

        "title": "Grocery Shopping",
        "amount": 1250.0,
        "category": "Food & Groceries",
        "date": "2026-10-01",
        "payment_method": "UPI / GPay",
        "note": "Vegetables, milk, and
snacks"
    },
    {
        "id": 2,
        "title": "Metro & Bus Card Recharge",
        "amount": 500.0,
        "category": "Transportation",
        "date": "2026-10-01",
        "payment_method": "Debit Card",
        "note": "Monthly commute pass"
    },
    {
        "id": 3,
        "title": "Python Programming Textbook",
        "amount": 650.0,
        "category": "Education",
        "date": "2026-09-28",
        "payment_method": "UPI / GPay",
        "note": "Reference book for college"
    },
    {
        "id": 4,
        "title": "High Speed WiFi Bill",
        "amount": 799.0,
        "category": "Bills & Utilities",
        "date": "2026-09-25",
        "payment_method": "Net Banking",
        "note": "Broadband monthly plan"
    },
    {
        "id": 5,
        "title": "Weekend Movie & Popcorn",
        "amount": 450.0,
        "category": "Entertainment",
        "date": "2026-09-21",
        "payment_method": "Cash",
        "note": "Cinema with friends"
    }
]

def show_expenses():
    print("\n----- ALL EXPENSES      ")

    for expense in expenses:
        print("\nID:", expense["id"])
        print("Title:", expense["title"])
        print("Amount: ₹", expense["amount"])
        print("Category:", expense["category"])
        print("Date:", expense["date"])
        print("Payment:", expense["payment_method"])
        print("Note:", expense["note"])

def total_expenses():
    total = 0

    for expense in expenses:
        total += expense["amount"]

    print("\nTotal Expenses: ₹", total)

def category_expenses():
    category = input("\nEnter category: ")

    total = 0
    found = False

    print("\n----- CATEGORY EXPENSES      ")

    for expense in expenses:
        if expense["category"].lower() == category.lower():

```

```

        found = True

        print("\nID:", expense["id"])
        print("Title:", expense["title"])
        print("Amount: ₹", expense["amount"])
        print("Category:", expense["category"])
        print("Date:", expense["date"])
        print("Payment:", expense["payment_method"])
        print("Note:", expense["note"])
        total += expense["amount"]

    if found:
        print("\nTotal for", category, ": ₹", total)
    else:
        print("\nNo expenses found for", category)

def add_expense():
    new_id = len(expenses) + 1

    title = input("Enter title: ")
    amount = float(input("Enter amount: "))
    category = input("Enter category: ")
    date = input("Enter date: ")
    payment = input("Enter payment method: ")
    note = input("Enter note: ")

    new_expense = {
        "id": new_id,
        "title": title,
        "amount":
            amount,
        "category": category,
        "date": date,
        "payment_method": payment,
        "note": note
    }

    expenses.append(new_expense)

    print("\nExpense added successfully!")

while True:

    print("\n===== EXPENSE TRACKER =====")
    print("1. Show all expenses")
    print("2. Show total expenses")
    print("3. Show category expenses")
    print("4. Add expense")
    print("5. Exit")
    choice = input("\nEnter your choice: ")
    if choice == "1":
        show_expenses()

    elif choice == "2":
        total_expenses()

    elif choice == "3":
        category_expenses()

    elif choice == "4":
        add_expense()

    elif choice == "5":
        print("Thank you!")
        break

    else:
        print("Invalid choice!")

```

4.4.1 Data Representation

The central data structure is the expenses list. Every expense is represented as a dictionary. The id field uniquely identifies the entry within the current list, title describes the expense, amount stores the monetary value as a floating-point number, category groups similar transactions, date records the transaction date, payment_method stores the payment channel, and note provides additional context.

4.4.2 Display and Aggregation

The show_expenses() function iterates through every dictionary using a for loop and prints the stored values. The total_expenses() function uses an accumulator variable named total. The algorithm is linear because every stored record is visited once. If n is the number of expenses, the time complexity of the total calculation is O(n), with O(1) additional working space.

4.4.3 Category Filtering

The category_expenses() function demonstrates searching and aggregation together. The category entered by the user is converted to lowercase and compared with the lowercase form of each stored category. This makes the search case-insensitive; for example, 'education' and 'Education' match. The found flag is used to distinguish between a successful search and a category with no records.

4.4.4 Adding a New Expense

The add_expense() function obtains a new ID using len(expenses) + 1, reads the required values from the user, constructs a dictionary with the same schema as the existing records, and appends it to the list. This keeps all records consistent with the original data representation.

4.4.5 Menu Control

The while True loop provides continuous interaction. The if/elif structure maps each menu option to its corresponding function. Selecting 5 triggers break and terminates the program. Invalid choices do not terminate the application; instead, an error message is printed and the menu is displayed again

5 Results and Discussion

The supplied sample dataset contains five expenses with a combined value of ₹3,649. The records cover five different categories, which makes the sample suitable for demonstrating both overall and category-level analysis. The program successfully represents each transaction, displays its details, computes the total, searches categories, and supports insertion of additional records during execution.

5.1 Sample Dataset Analysis

ID	Expense	Category	Amount	Payment Method
1	Grocery Shopping	Food & Groceries	₹1,250.00	UPI / GPay
2	Metro & Bus Card Recharge	Transportation	₹500.00	Debit Card
3	Python Programming Textbook	Education	₹650.00	UPI / GPay
4	High Speed WiFi Bill	Bills & Utilities	₹799.00	Net Banking
5	Weekend Movie & Popcorn	Entertainment	₹450.00	Cash

5.2 Expected Functional Results

Operation	Expected Result
Show all expenses	Displays all five records with ID, title, amount, category, date, payment method, and note.
Show total expenses	Displays Total Expenses: ₹3649.0 for the initial dataset.
Category: Education	Displays the Python Programming Textbook entry and category total of ₹650.0.
Category: Transportation	Displays the Metro & Bus Card Recharge entry and category total of ₹500.0.
Category not present	Displays a message stating that no expenses were found for the entered category.
Add expense	Creates a new dictionary record and appends it to the expenses list.
Exit	Displays 'Thank you!' and terminates the program.

5.3 Category-wise Interpretation

For the initial sample, Food & Groceries is ₹1,250, Bills & Utilities is ₹799, Education is ₹650, Transportation is ₹500, and Entertainment is ₹450. These values sum to ₹3,649. Since each sample transaction belongs to a different category, the category filter returns one record for each of the five initial categories. The current application performs aggregation on demand rather than storing precomputed totals. This reduces the risk of totals becoming inconsistent when a new expense is appended, because the total is recalculated from the current list whenever the user requests it.

5.4 Testing

Test Case	Input / Action	Expected Outcome	Status
TC01	Select option 1	All existing expenses are displayed	Pass
TC02	Select option 2	Overall total ₹3,649.00 is displayed	Pass
TC03	Enter category 'education'	Education expense is found despite lowercase input	Pass
TC04	Enter an unavailable category	No-expenses-found message is displayed	Pass
TC05	Select option 4 and enter a valid expense	New record is appended successfully	Pass
TC06	Enter menu choice outside 1–5	Invalid choice message is displayed	Pass
TC07	Select option 5	Program terminates after thank-you message	Pass

5.5 Limitations

- Expenses are stored only in memory; data is lost when the program exits.
- There is no edit or delete operation in the current implementation.

- The date is accepted as text, so the program does not validate date format.
- The amount is converted to float, but negative values and other invalid monetary inputs are not explicitly validated.
- The generated ID is based on the current list length and is suitable for the simple current implementation but not a persistent database design.
- The current version provides text output rather than charts or a graphical interface

Mapping POs and SDGs

PO No.	Program Outcome	Project Relevance
PO1	Engineering Knowledge	Applies fundamental programming, data-structure, and problem-solving concepts to build a working expense-management application.
PO2	Problem Analysis	Analyses the practical problem of unstructured expense recording and decomposes it into storage, search, aggregation, and input tasks.
PO3	Design/Development of Solutions	Designs a modular menu-driven solution with separate functions for displaying, calculating, filtering, and adding expenses.
PO4	Conduct Investigations of Complex Problems	Tests multiple input conditions including valid categories, unavailable categories, invalid menu choices, and newly added records.
PO5	Modern Tool Usage	Uses Python and a suitable development environment to implement, execute, test, and debug the application.
PO8	Individual and Collaborative Work	Supports structured development, review, testing, and documentation of the project.
PO9	Communication	Presents the problem, architecture, implementation, test results, limitations, and future scope in a structured technical report.
PO11	Life-long Learning	Provides a foundation for extending the application with databases, visualisation, validation, analytics, and GUI/web technologies.

Program Specific Outcomes (PSOs) Mapping

PSO No.	Program Specific Outcome	Project Relevance
PSO1	Problem-Solving and Programming Skills	Applies Python programming, lists, dictionaries, loops, and functions to analyse and solve the real-world problem of expense recording and aggregation.
PSO2	Software Design and Development	Designs and develops a modular, menu-driven command-line application with separate functions for display, calculation, filtering, and insertion, and tests it with sample data.
PSO3	Emerging Technologies and Future-Ready Solutions	Provides a foundation for extending the application with persistent storage, data analytics and visualisation, and GUI/web or mobile interfaces.

1.1 Sustainable Development Goals (SDGs) Mapping

SDG	Goal	Relevance to Project
SDG 4	Quality Education	The project provides practical programming experience and demonstrates how software can solve a common real-world problem.
SDG 8	Decent Work and Economic Growth	Developing a usable software application strengthens programming, testing, documentation, and problem-solving skills relevant to employment.
SDG 9	Industry, Innovation and Infrastructure	The project demonstrates software-based process improvement and provides a base for scalable digital financial-management tools.

Conclusion

The Expense Tracker project successfully demonstrates the design and implementation of a simple personal expense-management application using Python. The application stores structured expense records, displays them, calculates the overall expenditure, filters records by category, and allows new expenses to be added during execution. The use of functions improves modularity, while lists and dictionaries provide a straightforward representation of transaction data.

The project also demonstrates how fundamental programming constructs can be combined to solve a practical problem. The sample dataset produces a total expenditure of ₹3,649, and the category search provides useful subtotals for individual spending groups. The testing section shows that the main operations and menu controls behave as intended for the supplied implementation.

Although the current version is intentionally simple, it establishes a clear base for a more complete expense-management system. Its modular design makes it possible to add persistent data storage, validation, editing and deletion, budget monitoring, monthly reports, graphical charts, authentication, and web or mobile interfaces in future versions.

Bibliography

8. Python Software Foundation. Python 3 Documentation. Python Language Reference and Standard Library documentation.
9. Python Software Foundation. Python Data Structures — Lists, Dictionaries, Loops, and Functions.
10. Python Software Foundation. Built-in Types and Built-in Functions documentation.
11. GeeksforGeeks. Python Programming tutorials covering lists, dictionaries, functions, loops, and file handling.
12. W3Schools. Python tutorial and reference material for basic Python programming concepts.

Appendix A - Sample Console Flow

```
===== EXPENSE TRACKER =====
* Show all expenses
* Show total expenses
* Show category expenses
* Add expense
* Exit

Enter your choice: 2

Total Expenses: ₹ 3649.0
```

The above flow illustrates the overall-total operation using the initial five records. Similar menu interactions can be used to display all records, filter by category, add a new record, or exit.

— END OF REPORT —